University Lecture: Secure Flutter Apps with Firebase

(Authentication, OAuth, and App Check)

Part 1: The "Who" - Authentication Fundamentals

This initial module establishes the foundational theory of application security. Before implementing any solution, it is essential to define the core concepts of identity, the unique risks of the mobile environment, and the taxonomy of security factors.

Section 1.1: Defining Authentication vs. Authorization

The two most fundamental concepts in application security are Authentication (AuthN) and Authorization (AuthZ). These terms are often used interchangeably, but they represent distinct, sequential processes.

- Authentication (AuthN): Verifying Who You Are
  Authentication is the process of verifying a user's identity, ensuring that they are who they claim to be.1 It is the "gatekeeper" at the front door, responsible for checking a user's credentials (like a password or biometric scan) against a trusted record. It answers the question: "Is this user really who they say they are?"
- Authorization (AuthZ): Verifying What You Can Do
  Authorization is the process that follows authentication. It involves granting or denying permissions to an already authenticated user based on their verified identity.1 This is the "keymaster" inside the building, determining which specific doors a verified user is allowed to open. It answers the question: "Now that I know who this user is, what are they allowed to access?"

A critical security principle is that authorization *must* be preceded by authentication. For example, a banking app first authenticates a user with a password and Face ID (AuthN). Only after that successful authentication does the system check if that user is *authorized* to transfer funds from a specific account (AuthZ).[1] All our future work with Firebase Security Rules is a form of Authorization that is entirely dependent on the identity provided by Firebase Authentication (AuthN).

The Mobile Security Imperative

The need for robust authentication is particularly acute in the mobile ecosystem for three primary reasons:

1. **Sensitive Data Access:** Mobile devices are no longer secondary gadgets. They are the primary interface for users' most sensitive information, including financial (banking), health (healthcare), and personal (social media) records.[1] A security breach on a mobile app can be catastrophic.
2. **Personalization and Trust:** Modern users expect a seamless, personalized experience.[3] This personalization requires storing and accessing user-specific data. Secure authentication is the bedrock of the user trust required to provide this experience.
3. **Unique Threat Model:** The mobile threat vector is distinct from traditional web platforms. A user's phone can be lost or stolen, and it is frequently connected to insecure public Wi-Fi networks.[1] Weak authentication methods, such as a simple 4-digit PIN, are insufficient to protect against these risks.[1]

Section 1.2: A Taxonomy of Authentication Methods

Authentication relies on one or more "factors," which are distinct types of evidence used to prove an identity.[5] These factors are formally categorized into a security taxonomy.

- Factor 1: Knowledge ("Something you know")
  This is the most common factor, representing a piece of secret information that only the user should know. Examples include passwords, Personal Identification Numbers (PINs) 1, and security questions.1 This factor is also the weakest, as it is vulnerable to being forgotten, guessed (brute-force attacks), phished, or shared.1
- Factor 2: Possession ("Something you have")
  This factor is a unique physical or digital item in the user's possession. Examples include a physical USB security key 5, or, more commonly in mobile, a One-Time Password (OTP) generated by an authenticator app or sent via SMS/Email.1 This factor is stronger than knowledge alone but is still vulnerable; SMS OTPs, for instance, can be intercepted through SIM swapping attacks.1
- Factor 3: Inherence ("Something you are")
  This factor represents a unique biological trait of the user. Common implementations in mobile devices include fingerprint scanning, facial recognition (e.g., Apple's Face ID), and iris scanning.1

The security model for the Inherence factor is fundamentally superior. When a user authenticates with a password, that (hashed) password is sent to a server for verification. With biometrics, the verification "happens locally on the device".[6] Sensitive biometric data is *never* transmitted over the internet or stored on a server.[6] The device's secure enclave simply performs the check locally and sends a "yes" or "no" token to the application. This means that even if a company's servers are breached, attackers cannot steal the users' biometric data.

- The Gold Standard: Multi-Factor Authentication (MFA)

MFA is an authentication method that requires a user to successfully present evidence from two or more distinct factors.1 A common example is using a password (Knowledge) and an SMS OTP (Possession). MFA provides a layered defense based on the premise that an unauthorized actor is highly unlikely to be able to supply all required factors, such as stealing a user's password and their physical phone.5

Table 1.1: The Factors of Authentication

Factor Type	Description	Common Examples	Vulnerabilities
Knowledge	Something only you know.	Password, PIN, Security Question [1]	Brute-force, Phishing, Guessing [1]
Possession	Something only you have.	Phone (for OTP/SMS), USB Key, Auth App [1]	SIM Swapping, Interception [1], Theft
Inherence	Something unique you are.	Fingerprint, Face ID, Iris Scan [1]	(Data is secure, but the *factor* can be spoofed)
Location	Somewhere you are.	Geofencing (e.g., must be "in-office") [5]	VPNs, GPS Spoofing

Part 2: The "Service" - Firebase Authentication

Given the complexity of authentication, building a custom solution is a high-risk endeavor. This module introduces Firebase Authentication as a managed service and implements the most foundational provider.

Section 2.1: Why Not Build Your Own? The "Build vs. Buy" Dilemma

The "roll your own auth" fallacy is the dangerous belief that a development team can easily build a custom authentication system. This is a common but critical mistake.

- **Immense Complexity:** A secure authentication system is far more than an if password == hash check. It involves securely hashing passwords (e.g., bcrypt/scrypt), generating and managing cryptographic salts, secure user storage, token generation and validation (e.g., JWTs), token revocation, implementing email/phone verification flows, rate limiting to prevent brute-force attacks, and creating secure password reset mechanisms.[2]
- **Constant Maintenance:** Security is not a "set it and forget it" feature. It requires "significant effort" [2] and constant vigilance to "stay up to date with the latest security, protocol, and browser updates".[2] New vulnerabilities are discovered daily.
- **High Cost (Time & Money):** Building and maintaining this system is a full-time job for a

team of specialists.[8] Using a pre-built solution saves "lots of time".[9]

Using a Backend-as-a-Service (BaaS) like Firebase Authentication is a senior-level architectural decision that improves an app's security posture. It is an "auth as a service" [7] that provides an "audited and most importantly properly maintained cloud solution by a trusted provider".[8] This delegates the most complex and high-risk components to a dedicated team of experts, allowing the app's developers to "focus on the rest of your application".[2]

Section 2.2: The Firebase Authentication Ecosystem

The core Flutter plugin for all Firebase authentication operations is firebase_auth.[11] This single plugin provides an end-to-end identity solution [12] that supports a comprehensive list of providers. These providers can be grouped into three main categories [12]:

1. **Native Providers:** Methods managed directly by Firebase.
    - Email & Password
    - Email Link (Passwordless)
    - Phone Number (SMS)
2. **Federated / Social Providers (OAuth):** Allows users to sign in with existing accounts.
    - Google [12]
    - Apple [12]
    - Facebook, Twitter, GitHub, Microsoft, Yahoo [12]
3. **Other Methods:**
    - **Anonymous Authentication:** Creates a temporary, "guest" user account without requiring credentials. This is useful for onboarding flows or allowing users to try an app before committing to registration.[12]
    - **Custom Auth System:** Allows integration with an existing corporate or legacy authentication backend.[8]

Table 2.1: Firebase Authentication Providers (Flutter)

Category	Provider	Flutter Plugin(s) Required
Native	Email & Password	firebase_auth
Native	Email Link	firebase_auth
Native	Phone Number	firebase_auth
Federated (OAuth)	Google	firebase_auth, google_sign_in [11]
Federated (OAuth)	Apple	firebase_auth, sign_in_with_apple [11]
Federated (OAuth)	Facebook	firebase_auth,

		flutter_facebook_auth [11]
Anonymous	Anonymous Sign-In	firebase_auth
Custom	Custom Token	firebase_auth

Section 2.3: Core Implementation: Email & Password

This section provides the complete code for implementing the most common authentication provider.

- **Step 1: Project Setup (Flutter)**
  1. Install the core Firebase authentication plugin from the project root:
     flutter pub add firebase_auth 16
  2. Ensure Firebase is initialized in lib/main.dart before runApp() is called:
     Dart

```
// lib/main.dart
import 'package:firebase_core/firebase_core.dart';
//...

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // This line initializes the Firebase app
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(MyApp());
}
```

     (Based on [16])
- **Step 2: Project Setup (Firebase Console)**
  1. In the Firebase Console, navigate to the **Authentication** section.
  2. Click the **Sign-in method** tab.
  3. Click "Email/Password" and set the **Enable** switch to the "on" position, then save.[16]
- Step 3: Registration (Create a New User)
  The createUserWithEmailAndPassword method is a two-step operation: it first creates the user account in the Firebase backend and then automatically signs that user in to the local app, returning a UserCredential object.20
  Code Example (auth_service.dart):
  Dart

```
import 'package:firebase_auth/firebase_auth.dart';

class AuthService {
```

```
final FirebaseAuth _auth = FirebaseAuth.instance;

Future<String> registerUser({
  required String email,
  required String password,
}) async {
  try {
    // This is the core Firebase method
    UserCredential userCredential = await _auth.createUserWithEmailAndPassword(
      email: email,
      password: password,
    );

    // Optional: Update the user's display name
    await userCredential.user?.updateProfile(displayName: "New User");

    return "Success";
  } on FirebaseAuthException catch (e) {
    // Handle specific errors
    if (e.code == 'weak-password') {
      return 'The password provided is too weak.';
    } else if (e.code == 'email-already-in-use') {
      return 'The account already exists for that email.';
    }
    return e.message?? "An error occurred";
  } catch (e) {
    return e.toString();
  }
}
```

(Synthesized from [18])

- Step 4: Sign-In (Log in an Existing User)
  The signInWithEmailAndPassword method validates the user's credentials. If successful, it returns a UserCredential and updates the app's persistent authentication state.20
  Code Example (auth_service.dart):
  Dart

```
import 'package:firebase_auth/firebase_auth.dart';

class AuthService {
  //... (registration method from above)...

  Future<String> signInUser({
    required String email,
```

```
  required String password,
 }) async {
  try {
   // This is the core Firebase method
   await _auth.signInWithEmailAndPassword(
    email: email,
    password: password,
   );
   return "Success";
  } on FirebaseAuthException catch (e) {
   // Handle specific errors
   if (e.code == 'user-not-found') {
    return 'No user found for that email.';
   } else if (e.code == 'wrong-password') {
    return 'Wrong password provided for that user.';
   }
   return e.message?? "An error occurred";
  } catch (e) {
   return e.toString();
  }
 }
}
```

(Synthesized from [18])

The UserCredential object returned by both methods [20] is the critical "receipt" of a successful operation. It contains the User object, which in turn contains the uid. This uid is the unique, permanent identifier for the user and is the essential key for linking this authentication account to a user profile document in a database, as discussed in Part 3.2.

Section 2.4: Robust Error Handling

Authentication methods will frequently fail due to user error or network issues. Raw exceptions should never be shown to the user. All Firebase Authentication calls must be wrapped in a try...catch block, specifically designed to handle the FirebaseAuthException class.[23]

This exception object contains a code property (e.g., 'user-not-found') [23], which allows the developer to map technical errors to clean, user-friendly messages.[25]

- **Example: A User-Friendly Switch Statement**

  Dart

```
String getFirebaseAuthErrorMessage(FirebaseAuthException e) {
 switch (e.code) {
```

```
    case 'user-not-found':
      return 'No user found for that email.';
    case 'wrong-password':
      return 'Incorrect password. Please try again.';
    case 'email-already-in-use':
      return 'This email address is already in use.';
    case 'weak-password':
      return 'The password is too weak.';
    case 'invalid-email':
      return 'The email address is not valid.';
    case 'operation-not-allowed':
      return 'Email/password accounts are not enabled.';
    case 'requires-recent-login':
      return 'This operation is sensitive and requires recent authentication. Please log in
again.';
    default:
      return 'An unknown error occurred. Please try again later.';
  }
}

// How to use it:
// } on FirebaseAuthException catch (e) {
//   return getFirebaseAuthErrorMessage(e);
// }
```

(Based on [20])

Table 2.2: Common FirebaseAuthException Codes

Error Code (e.code)	When It Happens (Commonly)	User-Friendly Message
user-not-found	Sign-in: Email doesn't exist [25]	"No user found for that email."
wrong-password	Sign-in: Password is incorrect [25]	"Incorrect password. Please try again."
email-already-in-use	Registration: Email already registered [25]	"This email address is already in use."
weak-password	Registration: Password isn't strong enough [25]	"Password is too weak. Please use a stronger password."
invalid-email	Sign-in or Registration: Email is malformed [25]	"Please enter a valid email address."

requires-recent-login	Sensitive ops (e.g., delete()): User logged in too long ago [20]	"This operation is sensitive. Please log in again."
too-many-requests	Firebase quota exceeded (e.g., SMS) [23]	"Too many requests. Please try again later."

Part 3: Managing the User Lifecycle

Authentication is not a single event but a continuous *state*. This module covers the canonical Flutter pattern for managing this state and the architectural best practice for handling associated user data.

Section 3.1: The "Auth Gate": Listening to State

A common but flawed pattern for handling authentication is *imperative navigation*—calling Navigator.push(context, HomePage()) after a successful sign-in. This logic is brittle. It fails to handle cases where the user closes and re-opens the app (their auth state is persisted, but the app starts on the login page) or if their token is revoked in the background, leaving them "stuck" on a screen they should no longer see.[27]
The robust and declarative solution is to *listen* to the auth state, not *check* it. Firebase Authentication provides the FirebaseAuth.instance.authStateChanges() stream.[19] This stream is a powerful tool: it emits a new value (User? or null) *every time* the authentication state changes, whether from a login, a logout, or the initial app start-up when Firebase checks its persistent session.[19]
This stream is consumed using a StreamBuilder widget. This widget, often called an "Auth Gate" or "Auth Wrapper," should be placed at the root of the application (e.g., as the home property of MaterialApp). It listens to the authStateChanges() stream and declaratively builds the correct UI in response.[19] This "elegant solution" [30] simplifies all authentication flow logic into a single widget and guarantees the UI *always* reflects the true authentication state.

- Code Example: The AuthGate Widget
  This widget becomes the home of the MaterialApp.
  Dart
  ```
  // lib/auth_gate.dart
  import 'package:firebase_auth/firebase_auth.dart';
  import 'package:flutter/material.dart';
  import 'home_screen.dart';
  import 'login_screen.dart';

  class AuthGate extends StatelessWidget {
    const AuthGate({super.key});
  ```

```
@Override
Widget build(BuildContext context) {
 return StreamBuilder<User?>(
  // Listen to the auth state stream
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
   // 1. While waiting for connection, show a loading indicator
   if (snapshot.connectionState == ConnectionState.waiting) {
    return const Center(child: CircularProgressIndicator());
   }

   // 2. If the snapshot has data, a user is logged in
   if (snapshot.hasData) {
    // User is signed in, show the Home Screen
    // snapshot.data contains the User object
    return const HomeScreen();
   }

   // 3. If the snapshot has no data, user is logged out
   // User is signed out, show the Login Screen
   return const LoginScreen();
  },
 );
 }
}
```

(Synthesized from [17])

Section 3.2: Storing User Profile Data

A common point of confusion is where to store application-specific user data, such as a biography, a username, or "pro" status. The Firebase User object (from FirebaseAuth.instance.currentUser) is for *authentication*, not *application data*. While it can store a displayName and photoURL [21], it is impossible to "add arbitrary data to the Firebase Authentication user profile".[32]

The best practice is to **decouple the authentication profile from the database profile**:

1. Create a separate **users collection in Cloud Firestore**.
2. During user registration (inside the createUserWith... function), retrieve the uid from the returned UserCredential object.[21]
3. Use this uid as the **Document ID** for a new document within the users collection.

4. Store all additional, arbitrary profile data (e.g., bio, username, theme preference) inside this document.[33]

This architecture creates a clean 1:1 link between an Auth User and a Firestore Document, using the uid as the "foreign key."

Securing Your Profile Data: Firestore Security Rules

This newly created data must be secured. This is an act of **Authorization (AuthZ)**. Cloud Firestore Security Rules must be written to ensure a user can *only* read and write their *own* profile document.

Security rules provide a request.auth.uid variable, which contains the uid of the authenticated user making the request.[34] This variable is compared to the userId in the document path to enforce ownership.

- **Code Example (firestore.rules):**

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Match any document in the "users" collection
    // {userId} is a wildcard that captures the document ID
    match /users/{userId} {

      // A user can read, update, or delete THEIR OWN document
      allow read, update, delete: if request.auth!= null
                          && request.auth.uid == userId;

      // Any authenticated user can CREATE a user profile
      // (This is necessary for the sign-up process)
      allow create: if request.auth!= null;
    }
  }
}
```

(Synthesized from [34])

Part 4: The "Easy Button" - Implementing OAuth Providers

This module covers Federated Identity [2], which allows users to sign in with accounts they

already own (e.g., Google, Apple). The primary challenge in this area is not the Dart code, but the extensive, platform-specific native configuration.

Section 4.1: Google Sign-In (Android & iOS)

The OAuth flow is a two-step "credential handoff." First, the app authenticates with the *Provider* (Google) using their native SDK. Second, the app *passes the resulting credential* (an ID token) to *Firebase* (signInWithCredential), which verifies it and creates its own Firebase session token. Firebase *trusts* Google to have verified the user.[37]

- Step 1: Flutter Setup
  Install the necessary plugins:
  flutter pub add firebase_auth
  flutter pub add google_sign_in 19
- Step 2: Firebase Console Setup
  In Authentication -> Sign-in method, find and Enable the "Google" provider.19
- Step 3: Android Configuration (SHA-1 Key)
  This is the most critical and common point of failure. Google Sign-In on Android requires the app's signing certificate SHA-1 fingerprint to be registered.
  1. **Get the SHA-1 Key:** Open a terminal and navigate into the Flutter project's android directory: cd android.[39]
  2. Run the signing report command: ./gradlew signingReport.[39]
  3. **Copy the SHA-1:** From the output, find the debug variant and copy its SHA-1 fingerprint.[40]
  4. **Register the Key:** In the Firebase Console, go to **Project Settings** (the gear icon) -> **Your apps** -> Select your Android app. Click **"Add fingerprint"** and paste the SHA-1 key.[41]
- Step 4: Flutter Implementation (The Code)
  The implementation uses the google_sign_in package to initiate the flow and firebase_auth to complete it.
  Dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:google_sign_in/google_sign_in.dart';

class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;
  final GoogleSignIn _googleSignIn = GoogleSignIn();

  Future<UserCredential?> signInWithGoogle() async {
    try {
      // 1. Trigger the Google authentication flow
      final GoogleSignInAccount? googleUser = await _googleSignIn.signIn();
```

```
    // 2. Obtain the auth details (tokens) from the request
    if (googleUser == null) return null; // User cancelled
    final GoogleSignInAuthentication googleAuth = await googleUser.authentication;

    // 3. Create a new Firebase credential
    final AuthCredential credential = GoogleAuthProvider.credential(
      accessToken: googleAuth.accessToken,
      idToken: googleAuth.idToken,
    );

    // 4. Sign in to Firebase with the credential
    return await _auth.signInWithCredential(credential);
   } on FirebaseAuthException catch (e) {
    // Handle errors
    return null;
   }
  }
}
```

(Based on [19])

Section 4.2: Sign-In with Apple

This flow is conceptually identical to Google's but requires a significantly more complex, multi-stage configuration within the Apple Developer ecosystem.

- Step 1: Flutter Setup
  Install the necessary plugins:
  flutter pub add firebase_auth
  flutter pub add sign_in_with_apple 46
- **Step 2: Apple Developer Console Setup**
  1. Go to developer.apple.com -> "Certificates, Identifiers & Profiles."
  2. **Identifiers:** Select the app's Bundle ID.[46]
  3. **Capability:** Check the **"Sign In with Apple"** capability for this Bundle ID and save.[46]
  4. **Keys:** Create a new Private Key. Enable "Sign In with Apple" for it. Download the .p8 key file (this can only be downloaded once). Note the **Key ID**.[49]
  5. **Service ID:** Create a new "Services ID." This identifier will be used in the Firebase console.[47]
- **Step 3: Firebase Console Setup**
  1. In Authentication -> Sign-in method, **Enable** the "Apple" provider.[48]

2. Enter the "Service ID" created in Step 2.5.
3. Enter the **Apple Team ID** (found in the top-right of the Apple Developer account).
4. Upload the .p8 private key file and enter its **Key ID** (from Step 2.4).[49]

- Step 4: Xcode Configuration
  This step is mandatory and separate from the Developer Console setup.
  1. In the Flutter project, open the ios/Runner.xcworkspace file in Xcode.[47]
  2. Click "Runner" in the left navigator, then select the **"Signing & Capabilities"** tab.
  3. Click **+ Capability** and add **"Sign In with Apple"** from the list.[46]
- Step 5: Flutter Implementation (The Code)
  Apple Sign-In requires a nonce—a unique, random string—for replay protection. The crypto package (flutter pub add crypto) is used to generate and hash this nonce.37
  Dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:sign_in_with_apple/sign_in_with_apple.dart';
import 'package:crypto/crypto.dart' as crypto;
import 'dart:math';
import 'dart:convert';

class AuthService {
 final FirebaseAuth _auth = FirebaseAuth.instance;

 // Generates a cryptographically secure random nonce
 String _generateNonce([int length = 32]) {
  final charset =
'0123456789ABCDEFGHIJKLMNOPQRSTUVXYZabcdefghijklmnopqrstuvwxyz-._';
  final random = Random.secure();
  return List.generate(length, (_) => charset[random.nextInt(charset.length)]).join();
 }

 // Hashes the nonce for verification
 String _sha256ofString(String input) {
  final bytes = utf8.encode(input);
  final digest = crypto.sha256.convert(bytes);
  return digest.toString();
 }

 Future<UserCredential?> signInWithApple() async {
  try {
   final rawNonce = _generateNonce();
   final nonce = _sha256ofString(rawNonce);

   // 1. Trigger the Apple authentication flow
```

```
      final appleCredential = await SignInWithApple.getAppleIDCredential(
        scopes:,
        nonce: nonce, // Pass the hashed nonce
      );

      // 2. Create a new Firebase credential
      final AuthCredential credential = OAuthProvider("apple.com").credential(
        idToken: appleCredential.identityToken,
        rawNonce: rawNonce, // Pass the raw nonce for Firebase to verify
      );

      // 3. Sign in to Firebase with the credential
      return await _auth.signInWithCredential(credential);
    } catch (e) {
      // Handle errors
      return null;
    }
  }
}
```

(Based on [37])

Part 5: The "What" - Securing Your Backend with Firebase App Check

So far, security has focused on the *user* (AuthN) and their *data* (AuthZ). This final module introduces an advanced, additional layer of security: attestation, which protects the *app itself* and the *developer*.

Section 5.1: The "Other" Security Problem: App Impersonation

A critical threat vector exists that standard authentication cannot solve.

1. **The Threat:** A Flutter app bundles its Firebase configuration (google-services.json or GoogleService-Info.plist) within the app binary. These files contain API keys and database URLs.
2. **The Attack:** An attacker decompiles the app, extracts these keys, and writes a simple script (e.g., Python, Node.js) to make requests *directly* to the Firebase backend, completely bypassing the mobile app.
3. **The Impact:**

- **Billing Fraud:** The attacker can run a loop, creating millions of document writes or function calls, resulting in a massive, fraudulent bill for the developer.[53]
- **Data Poisoning:** The attacker can spam the database with "spurious requests" and malicious data, corrupting the application.[53]
- **Abuse:** The attacker can impersonate the app to phish for credentials or abuse backend resources.[55]

This attack **cannot be stopped by Security Rules alone**. If the attacker also uses a stolen username and password, their scripted request *will be authenticated*. The rule allow write: if request.auth.uid == userId; will **pass**, because the request is from a valid *user*, just not from the valid *app*.

Section 5.2: App Check vs. Authentication vs. Security Rules

Firebase App Check is the solution to this problem. It is "an additional layer of security" [55] that protects *the developer* from abuse.[53]
App Check answers one simple question: **"Is this request *really* coming from my genuine, unmodified app?"**.[53] It blocks all traffic that cannot *attest* to (i.e., prove) its authenticity.[56]
These three systems (App Check, Authentication, and Security Rules) create a "defense-in-depth" security model.

Table 5.1: The Three Layers of Firebase Security

Security Layer	Question It Answers	Who It Protects	How It Works
1. Firebase App Check	"Is this my *real, unmodified app*?"	**The Developer** (from abuse, billing fraud)	Attestation via Play Integrity or App Attest [53]
2. Firebase Auth	"Is this a *known, valid user*?"	**The User** (from unauthorized access)	Verifies user's identity via password, OAuth, etc. [57]
3. Security Rules	"Is this user *allowed to do this*?"	**The Data** (from misuse by valid users)	Authorization via request.auth.uid, resource.data, etc. [34]

Section 5.3: How App Check Works (Conceptually)

App Check "wraps" complex, platform-native attestation services into a simple Firebase API.[60]
The specific "attestation provider" depends on the platform:

- **Android:** Play Integrity API [61] (This replaces the older SafetyNet [53]).
- **iOS/macOS:** App Attest or DeviceCheck.[53]
- **Web:** reCAPTCHA v3 or reCAPTCHA Enterprise.[53]

The attestation flow operates as follows [57]:

1. **Attest:** The app (e.g., on Android) asks Play Integrity: "Am I a genuine app on a real, untampered device?".[58] Play Integrity provides a signed attestation "receipt."
2. **Verify & Exchange:** The App Check SDK sends this receipt to the Firebase App Check server.
3. **Get Token:** The App Check server verifies the receipt and, if valid, returns a short-lived **App Check Token** to the app.
4. **Send Token:** The App Check client SDK caches this token [57] and automatically includes it in the header of *all* future requests to Firebase services (e.g., Firestore, Storage, Cloud Functions).
5. **Block/Allow:** The Firebase backend service receives the request, checks for a valid App Check Token, and **blocks** the request at the "firewall" if the token is missing or invalid.[60]

Part 6: The "Firewall" - App Check & Security Rules Integration

This final module covers the practical implementation of App Check in a Flutter project, including the critical workflow for handling development versus production environments.

Section 6.1: Implementing App Check in Flutter

- **Step 1: Setup**
    1. Install the App Check plugin: flutter pub add firebase_app_check.[64]
    2. In the Firebase Console, go to **App Check**. For *each* app (Android, iOS), click to register the attestation providers. For Android, enable Play Integrity.[62] For iOS, enable App Attest.[63]
- Step 2: Activation in main.dart (Production)
  This code must run after Firebase.initializeApp() but before any other Firebase service (like Firestore) is called.64
  Dart

```
// lib/main.dart
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_app_check/firebase_app_check.dart';

void main() async {
```

```
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  // Activate App Check for production
  await FirebaseAppCheck.instance.activate(
    // For Android, use Play Integrity
    androidProvider: AndroidProvider.playIntegrity,

    // For iOS, use App Attest
    appleProvider: AppleProvider.appAttest,

    // For Web, use reCAPTCHA v3 (key from Google Cloud Console)
    webProvider: ReCaptchaV3Provider('YOUR_RECAPTCHA_V3_SITE_KEY'),
  );

  runApp(MyApp());
}
```

(Synthesized from [64])

Section 6.2: The Development Problem: Emulators & Simulators

There is a critical "Catch-22" when implementing App Check. The production providers (Play Integrity, App Attest) are designed to only validate real, physical devices.
Your Android Emulator and iOS Simulator are not real devices.67
Therefore, as soon as App Check is activated, **all requests from your development environment will be blocked**.[67]
The solution is the DebugProvider, which requires a manual, one-time setup for each development machine and emulator.

- **The Development Workflow (Mandatory):**
    1. **Modify main.dart (Temporarily):** Change the activate call to use the debug providers.
       Dart
       ```
       // Use this for development ONLY
       await FirebaseAppCheck.instance.activate(
         androidProvider: AndroidProvider.debug, // Use Debug provider
         appleProvider: AppleProvider.debug,   // Use Debug provider
       );
       ```

(Based on [65])

2. **Run the App:** Launch the app on the emulator/simulator.
3. Find the Token: Look in the debug console (e.g., the "Run" tab). App Check will print a log message containing a debug token.67
   D/DebugAppCheckProvider(12345): Enter this debug secret into the allow list in the Firebase Console for your project:
   [123a4567-b89c-12d3-e456-789012345678]
4. **Register the Token:**
   - Copy this 123a4567-b89c-12d3-e456-789012345678 token.
   - Go to Firebase Console -> App Check -> Your App -> "Manage debug tokens."
   - Click "Add debug token" and paste the token.[67]
5. **Restart the app.** Requests from that specific emulator will now be accepted.
6. **Warning:** The debug provider *must never* be used in a production build.[67] Build flavors or environment variables should be used to switch between AndroidProvider.debug and AndroidProvider.playIntegrity.

Section 6.3: Tying It All Together: Final Security Rules

After the app (with the App Check SDK) has been distributed, the final step is to go to the Firebase Console -> App Check -> **Products**. Here, select services like Firestore and click the **"Enforce"** button.[60]
This changes the order of operations for all backend requests:

1. A request arrives at the Firebase backend.
2. **Layer 1 (App Check):** The service checks for a valid App Check token. If it is missing or invalid, the request is **BLOCKED**.
3. **Layer 2 (Security Rules):** *Only if Layer 1 passes* does the request proceed to be evaluated by the Cloud Firestore Security Rules.[60]

This has a powerful implication: App Check *simplifies* the Security Rules. Because App Check runs *before* the rules [60], any request that reaches the rules is *already guaranteed* to have come from the genuine app. It is redundant to check for request.app!= null in the rules.
The rules can now be simpler and focus *only* on **User Authorization (AuthZ)**.

- The Final, Secure Architecture
  This architecture provides true defense-in-depth:
  1. **App Check** (activate() in main.dart + "Enforce" in console) verifies it's **your app**.
  2. **Auth State** (authStateChanges() + StreamBuilder) verifies it's a **logged-in user**.
  3. **Security Rules** (request.auth.uid == userId) verifies it's the *correct* user for that *specific* piece of data.
- **The Final Rule (Example):**
  rules_version = '2';
  service cloud.firestore {

```
 match /databases/{database}/documents {

  // This rule is simple, clean, and focuses on USER authorization.
  // It is automatically protected by App Check (APP attestation),
  // which runs before this rule is ever evaluated.
  match /users/{userId} {
   allow read, write: if request.auth!= null
                && request.auth.uid == userId;
  }
 }
}
```

(Based on [34])

Lucrări citate

1. Mobile Authentication: Everything You Need to Know - LoginRadius, accesată pe noiembrie 15, 2025, https://www.loginradius.com/blog/identity/mobile-authentication
2. Authentication and Authorization - Azure App Service | Microsoft Learn, accesată pe noiembrie 15, 2025, https://learn.microsoft.com/en-us/azure/app-service/overview-authentication-authorization
3. What is Secure Mobile Authentication? - Daon, accesată pe noiembrie 15, 2025, https://www.daon.com/resource/mobile-authentication-101/
4. Toward secure mobile applications through proper authentication mechanisms - PMC, accesată pe noiembrie 15, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC11620593/
5. Multi-factor authentication - Wikipedia, accesată pe noiembrie 15, 2025, https://en.wikipedia.org/wiki/Multi-factor_authentication
6. 8 Mobile Authentication Best Practices for 2025 - NextNative, accesată pe noiembrie 15, 2025, https://nextnative.dev/blog/mobile-authentication-best-practices
7. Firebase auth, vs my own pros/cons of having my users in firebase - Google Groups, accesată pe noiembrie 15, 2025, https://groups.google.com/g/firebase-talk/c/D9iPtgNELWg
8. Why would I use a custom token authentication for firebase instead of using my own auth solution? - Reddit, accesată pe noiembrie 15, 2025, https://www.reddit.com/r/Firebase/comments/1fswdgl/why_would_i_use_a_custom_token_authentication_for/
9. Firebase v Custom Backend : r/FlutterDev - Reddit, accesată pe noiembrie 15, 2025, https://www.reddit.com/r/FlutterDev/comments/175j4cj/firebase_v_custom_backend/

10. Firebase vs. Custom Backend: Choosing the Right Fit - KNGURU Studios, accesată pe noiembrie 15, 2025, https://www.knguru.de/en/blog/firebase-vs-benutzerdefiniertes-backend
11. Top Flutter User Authentication and Social Sign-in packages, accesată pe noiembrie 15, 2025, https://fluttergems.dev/authentication/
12. Firebase Authentication, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/auth
13. Flutter x Firebase | Authentication | by Jackie Moraa - Medium, accesată pe noiembrie 15, 2025, https://kymoraa.medium.com/flutter-x-firebase-authentication-a54e7ab40790
14. Authenticate with Firebase Anonymously on Apple Platforms, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/auth/ios/anonymous-auth
15. iOS Firebase Authentication: Sign In Anonymously Tutorial - YouTube, accesată pe noiembrie 15, 2025, https://www.youtube.com/watch?v=M7VEceh2U7Q
16. Get Started with Firebase Authentication on Flutter, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/auth/flutter/start
17. How to Build a Secure User Authentication Flow in Flutter with Firebase and Bloc State Management - freeCodeCamp, accesată pe noiembrie 15, 2025, https://www.freecodecamp.org/news/user-authentication-flow-in-flutter-with-firebase-and-bloc/
18. Firebase Authentication — Flutter | by Abhishek Doshi | Google Developer Experts, accesată pe noiembrie 15, 2025, https://medium.com/google-developer-experts/firebase-authentication-flutter-80e8f00338ac
19. Add a user authentication flow to a Flutter app using FirebaseUI | Firebase Codelabs, accesată pe noiembrie 15, 2025, https://firebase.google.com/codelabs/firebase-auth-in-flutter-apps
20. Using Firebase Authentication | FlutterFire, accesată pe noiembrie 15, 2025, https://firebase.flutter.dev/docs/auth/usage/
21. Using Flutter to Implement User Registration with Email, Password, and Username - Medium, accesată pe noiembrie 15, 2025, https://medium.com/@python-javascript-php-html-css/implementing-user-registration-with-email-password-and-username-in-flutter-a66944e217c3
22. Firebase Authentication with Email and Password in Flutter | HackerNoon, accesată pe noiembrie 15, 2025, https://hackernoon.com/authenticate-flutter-with-email-and-password-in-flutter
23. Error Handling | Firebase Documentation - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/auth/flutter/errors
24. How to Handle Firebase Auth exceptions on flutter - Stack Overflow, accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/56113778/how-to-handle-firebase-auth-exceptions-on-flutter
25. what are the error codes for Flutter Firebase Auth Exception? - Stack ..., accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/67617502/what-are-the-error-codes-for-flut

ter-firebase-auth-exception
26. Handling Firebase Authentication Errors in Flutter #13499 - GitHub, accesată pe noiembrie 15, 2025, https://github.com/firebase/flutterfire/discussions/13499
27. FirebaseAuth.instance.onAuthStateChanged is not triggering the stream builder when user logs out - Stack Overflow, accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/62703109/firebaseauth-instance-onauthstatechanged-is-not-triggering-the-stream-builder-wh
28. Understanding FirebaseAuth.instance Stream Methods: authStateChanges, userChanges, and idTokenChanges | by Robsen Shemsedin | Medium, accesată pe noiembrie 15, 2025, https://medium.com/@shemsedinrobsen/understanding-firebaseauth-instance-b7d7e5add5a0
29. Super Simple Authentication Flow with Flutter & Firebase | by ..., accesată pe noiembrie 15, 2025, https://medium.com/coding-with-flutter/super-simple-authentication-flow-with-flutter-firebase-737bba04924c
30. Flutter & Firebase authentication with streams and StreamBuilder - YouTube, accesată pe noiembrie 15, 2025, https://www.youtube.com/watch?v=nxu4bMpPvCQ
31. StreamBuilder & onAuthStateChange : r/Supabase - Reddit, accesată pe noiembrie 15, 2025, https://www.reddit.com/r/Supabase/comments/15ynbfk/streambuilder_onauthstatechange/
32. flutter - Firebase: should I store user data on the authentication ..., accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/56817428/firebase-should-i-store-user-data-on-the-authentication-profile-or-database
33. The Ultimate Guide to Flutter User Profile Page with Firebase - DhiWise, accesată pe noiembrie 15, 2025, https://www.dhiwise.com/post/flutter-user-profile-page-with-firebase-a-practica-guide
34. Writing conditions for Cloud Firestore Security Rules - Firebase, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/firestore/security/rules-conditions
35. Firestore Rules, request.auth.uid is not the same as uid I get from firebase auth google signin - Stack Overflow, accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/64853423/firestore-rules-request-auth-uid-is-not-the-same-as-uid-i-get-from-firebase-aut
36. Firestore Rules: How to Restrict Reads/Writes from Auth and Firebase Uid - YouTube, accesată pe noiembrie 15, 2025, https://www.youtube.com/watch?v=rIvskxFO6Qg
37. Social Authentication | FlutterFire, accesată pe noiembrie 15, 2025, https://firebase.flutter.dev/docs/auth/social/
38. Google Sign In With Firebase in Flutter - GeeksforGeeks, accesată pe noiembrie 15, 2025,

https://www.geeksforgeeks.org/firebase/flutter-google-sign-in-ui-and-authentication/
39. Generate SHA-1 for Flutter/React-Native/Android-Native app - Stack ..., accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/51845559/generate-sha-1-for-flutter-react-native-android-native-app
40. Client authentication | Google Play services, accesată pe noiembrie 15, 2025, https://developers.google.com/android/guides/client-auth
41. firebase - How to add SHA-1 to android application - Stack Overflow, accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/39144629/how-to-add-sha-1-to-android-application
42. How to Get SHA-1 & SHA-256 Keys for Firebase | Android Studio & Keytool Methods (2025), accesată pe noiembrie 15, 2025, https://www.youtube.com/watch?v=ndlCRoLogWA
43. "Secure Flutter Authentication with Firebase: A Comprehensive Guide to Login, Register, and SHA Key Integration" | by Kavy mistry | Medium, accesată pe noiembrie 15, 2025, https://medium.com/@kavyamistry0612/secure-flutter-authentication-with-firebase-a-comprehensive-guide-to-login-register-and-sha-key-3b68f2297938
44. Flutter — Google Sign In using firebase authentication[step-by-step] | by DevLens - Medium, accesată pe noiembrie 15, 2025, https://medium.com/@dev.lens/flutter-google-sign-in-using-firebase-authentication-step-by-step-ef2ddfb84a2c
45. Google Sign-In in Flutter with Firebase Authentication - YouTube, accesată pe noiembrie 15, 2025, https://www.youtube.com/watch?v=MMI-tpnlijg
46. Flutter Firebase Authentication: Apple Sign In - DEV Community, accesată pe noiembrie 15, 2025, https://dev.to/offlineprogrammer/flutter-firebase-authentication-apple-sign-in-1m64
47. Integrating Sign In with Apple in Flutter: Privacy-First Authentication for Flutter App - Medium, accesată pe noiembrie 15, 2025, https://medium.com/@dtejaswini.06/integrating-sign-in-with-apple-in-flutter-privacy-first-authentication-for-flutter-app-662df02a03e4
48. Apple Login | FlutterFlow Documentation, accesată pe noiembrie 15, 2025, https://docs.flutterflow.io/integrations/authentication/firebase/apple/
49. Authenticate Using Apple | Firebase - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/auth/ios/apple
50. nickmeinhold/apple-sign-in-flutter-firebase: Example project to document using the sign_in_with_apple plugin and firebase auth. - GitHub, accesată pe noiembrie 15, 2025, https://github.com/nickmeinhold/apple-sign-in-flutter-firebase
51. Build and release an iOS app - Flutter documentation, accesată pe noiembrie 15, 2025, https://docs.flutter.dev/deployment/ios
52. Android Sign in with apple and firebase flutter - Stack Overflow, accesată pe noiembrie 15, 2025,

https://stackoverflow.com/questions/62805312/android-sign-in-with-apple-and-firebase-flutter
53. Firebase App Check: A New Dimension in App Security | by Hiranya Jayathilaka - Medium, accesată pe noiembrie 15, 2025, https://medium.com/firebase-developers/firebase-app-check-a-new-dimension-in-app-security-96c807978ae
54. What is the purpose of Firebase AppCheck? - Stack Overflow, accesată pe noiembrie 15, 2025, https://stackoverflow.com/questions/67948531/what-is-the-purpose-of-firebase-appcheck
55. Firebase App Check | Protect your backend APIs - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/products/app-check
56. accesată pe noiembrie 15, 2025, https://firebase.google.com/products/app-check#:~:text=App%20Check%20is%20an%20additional,that%20has%20not%20been%20attested.
57. Firebase App Check - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check
58. accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check#:~:text=It%20works%20with%20both%20Google,originate%20from%20your%20authentic%20app
59. App Check | React Native Firebase, accesată pe noiembrie 15, 2025, https://rnfirebase.io/app-check/usage
60. Introducing Firebase App Check - YouTube, accesată pe noiembrie 15, 2025, https://www.youtube.com/watch?v=LFz8qdF7xg4
61. Overview of the Play Integrity API - Android Developers, accesată pe noiembrie 15, 2025, https://developer.android.com/google/play/integrity/overview
62. Get started using App Check with Play Integrity on Android - Firebase - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check/android/play-integrity-provider
63. Get started using App Check with App Attest on Apple platforms - Firebase - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check/ios/app-attest-provider
64. Get started using App Check in Flutter apps | Firebase Documentation - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check/flutter/default-providers
65. firebase_app_check example | Flutter package - Pub.dev, accesată pe noiembrie 15, 2025, https://pub.dev/packages/firebase_app_check/example
66. Enable App Check in Flutter apps, accesată pe noiembrie 15, 2025, https://firebase.flutter.dev/docs/app-check/default-providers/
67. Use App Check with the debug provider with Flutter - Firebase - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check/flutter/debug-provider
68. Use App Check with the debug provider on Android - Firebase - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check/android/debug-provider

69. Use App Check with the debug provider on Apple platforms - Firebase - Google, accesată pe noiembrie 15, 2025, https://firebase.google.com/docs/app-check/ios/debug-provider
70. Use App Check with the debug provider - FlutterFire, accesată pe noiembrie 15, 2025, https://firebase.flutter.dev/docs/app-check/debug-provider/
71. Use App Check with the debug provider | Sign in with Google for iOS, accesată pe noiembrie 15, 2025, https://developers.google.com/identity/sign-in/ios/appcheck/debug-provider
72. firestore security rules with app check : r/Firebase - Reddit, accesată pe noiembrie 15, 2025, https://www.reddit.com/r/Firebase/comments/1gyuvvg/firestore_security_rules_with_app_check/